

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

Кафедра интеллектуально информационных технологий

Отчет по лабораторной работе №7

Специальность ПО-12

Выполнил:

Середич К. Н.

Студент группы ПО-12

Проверила:

Кулик А. Д.

Преподаватель-стажер

кафедры ИИТ

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений.

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
- Организовать ввод параметров для создания объектов (использовать экранные компоненты);
- Осуществить визуализацию графических примитивов

Важное замечание: должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию.

Для всех динамических сцен необходимо задавать параметр скорости!

11) Изобразить прямоугольник, вращающийся в плоскости формы вокруг одной из своих вершин.

Входные данные: Вводятся пользователем через графический интерфейс (текстовые поля `ttk.Entry`).

- Ширина и Высота прямоугольника (вещественные числа > 0).
- Скорость вращения в градусах в секунду (вещественное число).
- Управляющие команды (кнопки): «Применить», «Пауза/Продолжить», «Скриншот».

Выходные данные:

- Визуальный: Динамическая анимация на виджете `tk.Canvas` (прямоугольник, вращающийся вокруг координаты 0,0, перенесенной в центр холста).
- Файловый: При нажатии на кнопку «Скриншот» генерируется векторное изображение в формате `.eps` (например, `lab7_task1_168...eps`) и сохраняется в текущую рабочую директорию.

```
import math
import os
import time
import tkinter as tk
from tkinter import messagebox, ttk

class RotatingRectangle:
    def __init__(self, width: float, height: float, angle_rad: float = 0.0) -> None:
        self.width = max(width, 1e-6)
        self.height = max(height, 1e-6)
        self.angle_rad = angle_rad

    def corners(self) -> tuple[tuple[float, float], ...]:
        w, h = self.width, self.height
        base = ((0.0, 0.0), (w, 0.0), (w, h), (0.0, h))
        c, s = math.cos(self.angle_rad), math.sin(self.angle_rad)
        rotated = []
        for x, y in base:
            xr = x * c - y * s
            yr = x * s + y * c
            rotated.append((xr, yr))
        return tuple(rotated)

class RotatingRectangleApp(tk.Tk):
    def __init__(self) -> None:
        super().__init__()
        self.title("ЛР7 – задание 1: вращающийся прямоугольник")
        self.geometry("720x560")
        self._rect = RotatingRectangle(120.0, 80.0, 0.0)
```

```

self._anim_job = None
self._paused = False

# ... (инициализация элементов UI: Entry, Button, Canvas) ...
self._canvas = tk.Canvas(self, bg="white", highlightthickness=1)
self._canvas.pack(fill=tk.BOTH, expand=True, padx=8, pady=8)
self.bind("<Configure>", lambda e: self._draw())
self._schedule_frame()

def _draw(self) -> None:
    self._canvas.delete("all")
    cx, cy = self.winfo_width() / 2.0, self.winfo_height() / 2.0
    corners = self._rect.corners()
    flat = []
    for x, y in corners:
        flat.extend((cx + x, cy - y))
    self._canvas.create_polygon(*flat, outline="blue", fill="#cfe8ff", width=2)
    self._canvas.create_oval(cx-5, cy-5, cx+5, cy+5, outline="red", fill="red")

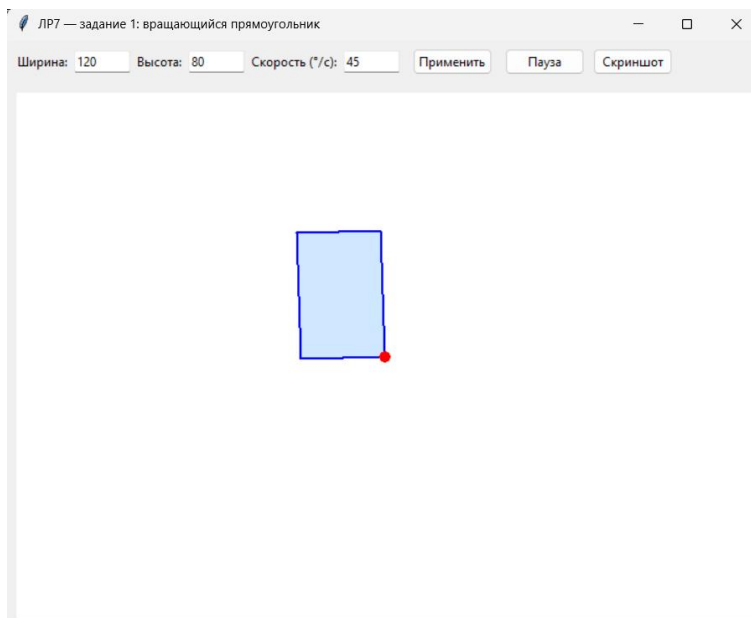
def _tick(self) -> None:
    if self._paused: return
    dps = float(self._var_speed.get().replace(",", "."))
    self._rect.angle_rad += math.radians(dps * 0.033)
    self._draw()
    self._schedule_frame()

def _schedule_frame(self) -> None:
    if self._anim_job: self.after_cancel(self._anim_job)
    if not self._paused:
        self._anim_job = self.after(33, self._tick)

def _screenshot(self) -> None:
    path = os.path.join(os.getcwd(), f"lab7_task1_{int(time.time() * 1000)}.eps")
    self._canvas.postscript(file=path, colormode="color", width=self._canvas.winfo_width())
    messagebox.showinfo("Скриншот", f"Сохранено:\n{path}")

if __name__ == "__main__":
    RotatingRectangleApp().mainloop()

```



Задание 2. Реализовать построение заданного типа фрактала по варианту Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

11) Множество Жюлиа

Входные данные: Вводятся пользователем через текстовые поля.

- $\text{Re}(c)$ и $\text{Im}(c)$ — действительная и мнимая части комплексной константы c (вещественные числа).
- `max_iter` — максимальное количество итераций (целое число > 0).
- Масштаб — радиус отображаемой области на комплексной плоскости (вещественное число > 0).
- Ширина и Высота итогового изображения в пикселях (целые числа).

Выходные данные:

- Визуальный: Отрисовка сгенерированного растрового изображения фрактала (цвет зависит от количества шагов до выхода за границу) в окне приложения через `tk.PhotoImage`.
- Файловый: Сохранение сгенерированного файла изображения в бинарном формате P6 PPM (.ppm) в текущую директорию.

```
import math
import os
import tempfile
import time
from dataclasses import dataclass
import tkinter as tk
from tkinter import messagebox, ttk

@dataclass(frozen=True)
class JuliaView:
    width: int; height: int
    xmin: float; xmax: float
    ymin: float; ymax: float
    max_iter: int; escape_bound: float

class JuliaSet:
    def __init__(self, c: complex, view: JuliaView) -> None:
        self.c = c
        self.view = view

    def ppm_bytes(self) -> bytes:
        w, h = self.view.width, self.view.height
        header = f"P6\n{w} {h}\n255\n".encode("ascii")
        dx = (self.view.xmax - self.view.xmin) / max(w - 1, 1)
        dy = (self.view.ymax - self.view.ymin) / max(h - 1, 1)
        body = bytearray(w * h * 3)
        idx = 0
        for j in range(h):
            im = self.view.ymax - j * dy
            for i in range(w):
                z = complex(self.view.xmin + i * dx, im)
                steps = 0
                while abs(z) <= self.view.escape_bound and steps < self.view.max_iter:
                    z = z * z + self.c
                    steps += 1
                # Генерация цвета
                if steps >= self.view.max_iter: r = g = b = 0
                else:
                    t = steps / self.view.max_iter
                    r = int(255 * (0.5 + 0.5 * math.sin(6.28 * t)))
                    g = int(255 * (0.5 + 0.5 * math.sin(6.28 * t + 2.1)))
                    b = int(255 * (0.5 + 0.5 * math.sin(6.28 * t + 4.2)))
                body[idx] = max(0, min(255, r))
                body[idx+1] = max(0, min(255, g))
                body[idx+2] = max(0, min(255, b))
                idx += 3
        return header + bytes(body)

class JuliaApp(tk.Tk):
    def __init__(self) -> None:
        super().__init__()
        self.title("ЛР7 — задание 2: множество Жюлиа")
        # ... (инициализация UI для ввода c, max_iter, размера) ...
        self._label = ttk.Label(self); self._label.pack()
        self._last_ppm: bytes | None = None

    def _render(self) -> None:
```

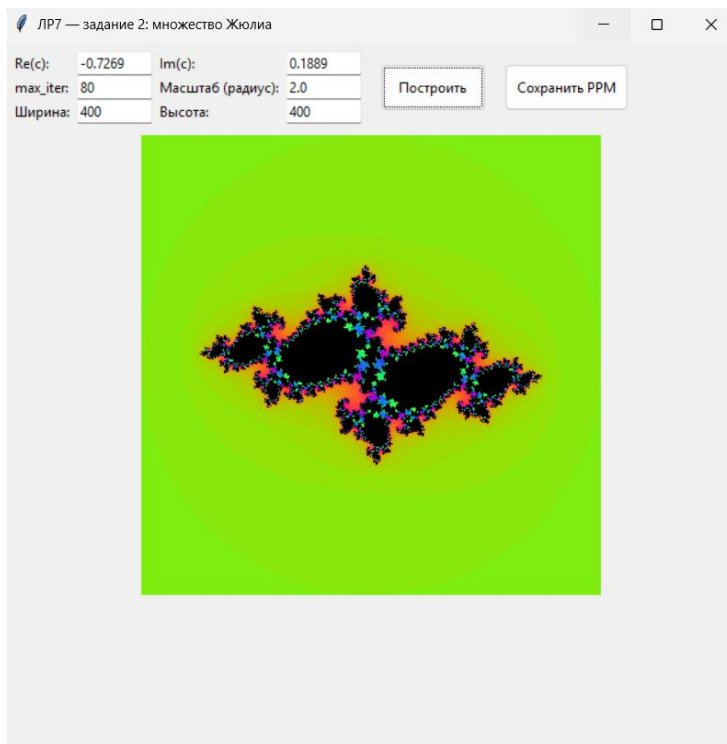
```

        c = complex(float(self._var_re.get()), float(self._var_im.get()))
        view = JuliaView(int(self._var_w.get()), int(self._var_h.get()), -2.0, 2.0, -2.0, 2.0,
int(self._var_iter.get()), 2.0)
        self.config(cursor="watch"); self.update_idletasks()
        ppm = JuliaSet(c, view).ppm_bytes()
        self._last_ppm = ppm
        self.config(cursor="")

        # Отображение через временный файл для совместимости
        fd, tmp_path = tempfile.mkstemp(suffix=".ppm")
        with os.fdopen(fd, "wb") as tmp_file: tmp_file.write(ppm)
        self._photo = tk.PhotoImage(master=self, file=tmp_path)
        self._label.configure(image=self._photo)
        os.unlink(tmp_path)

if __name__ == "__main__":
    JuliaApp().mainloop()

```



Вывод: мы освоили возможности языка программирования Python в разработке оконных приложений.